微算機系統實習

第12組專案報告

LAB 09

組別： 12
班級、姓名與學號：
資工三 112590450 李馥亘
資工三 112590452 林希芸
資工三 112590454 林妍蓁
資工三 112590455 吳哲丞
日期：2025.12.17

1. 實驗內容：

- 學習使用 Pthreads 建立與管理多執行緒
- 理解並實作任務平行 (Task Parallelism)與資料平行 (Data Parallelism)
- 使用 Semaphore 控制執行緒之間的訊號通知
- 使用 Mutex 保護 Critical Section，避免共享資源競爭
- 透過 GPIO 控制 LED，將執行緒狀態以視覺方式呈現

2. 實驗過程及結果

實驗過程

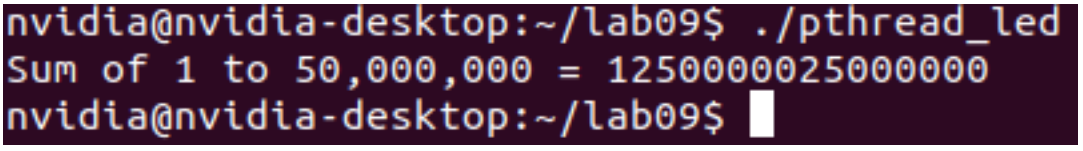
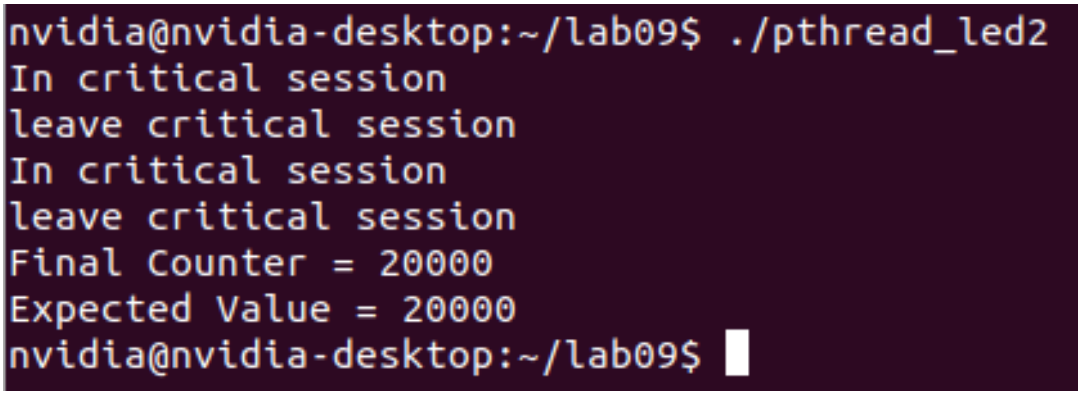
- 項目一（main.cpp）：多執行緒計算與狀態監控
    - 使用 Pthreads 建立三個執行緒，包含兩個 Worker Thread 與一個 Monitor Thread
    - Worker Threads 採用資料平行（Data Parallelism）方式，分別負責計算 1～25,000,000 與 25,000,001～50,000,000 的加總
    - 主執行緒於兩個 Worker Thread 完成後，將計算結果進行合併輸出
    - Monitor Thread 採用任務平行（Task Parallelism），於計算期間控制 LED1 以固定頻率閃爍
    - 使用 Semaphore 作為同步機制，在計算完成後通知 Monitor Thread 停止閃爍，並將 LED1 設為恆亮
- 項目二（main2.cpp）：共享資源保護與 Mutex 應用
    - 宣告全域變數 Counter 作為共享資源，並建立兩個執行緒同時進行累加操作
    - 使用 pthread mutex 將 Critical Section 包覆，避免多執行緒同時存取造成 Race Condition
    - 每次進入 Critical Section 時點亮 LED2，離開時熄滅 LED2，以視覺化 Mutex 鎖定與釋放狀態

預期實驗結果

- 項目一中 LED1 於計算期間持續閃爍，計算完成後轉為恆亮
- 系統能正確輸出 1～50,000,000 的加總結果
- 項目二中 Counter 最終結果為 20,000

- LED2 僅於進入 Critical Section 時亮起
- 系統運作過程中不發生資料競爭或計數錯誤

實際上的結果

項目一

項目二

項目一：多執行緒計算與監控
項目二：共享資源保護與 Mutex

遇到的問題&問題如何解決

- **Worker Threads 計算速度過快，LED 未能顯示閃爍狀態**
  - Worker Threads 的運算速度非常快，導致 Monitor Thread 尚未明顯進入 LED 閃爍狀態前，整個計算流程即已完成，使 LED 幾乎直接進入結束狀態，難以透過視覺方式確認運算是否正在進行。
  - 解決方法：為了便於觀察執行緒運作與計算流程，在 Worker Threads 開始運算前加入延遲（約 5,000,000 毫秒），讓 Monitor Thread 有足夠時間控制 LED 進行閃爍，進而透過 LED 狀態清楚判斷運算是否成功執行。

3. 程式碼：

項目一：main.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

using namespace std;

/* =======================
  GPIO 控制函式
======================= */
bool file_exists(const string &path) {
   ifstream f(path);
   return f.good();
}

void gpio_export(int pin) {
   if (!file_exists("/sys/class/gpio/gpio" +
to_string(pin))) {
       ofstream exportFile("/sys/class/gpio/export");
       if (exportFile.is_open()) {
           exportFile << pin;
           exportFile.close();
       }
   }
}

void gpio_set_direction(int pin, const string &dir) {
   string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/direction";
   ofstream dirFile(path);
```

```
    if (dirFile.is_open()) {
        dirFile << dir;
        dirFile.close();
    }
}

void gpio_set_value(int pin, int value) {
    string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/value";
    ofstream valFile(path);
    if (valFile.is_open()) {
        valFile << value;
        valFile.close();
    }
}

/* ======================
   全域變數
====================== */
long long result_T1 = 0;
long long result_T2 = 0;

sem_t sem_done; // 用來通知 Monitor Thread 停止閃爍
bool workers_finished = false;

int LED_HEARTBEAT = 396;    // LED1 (Heartbeat)

/* ======================
   Worker Thread 函式
====================== */
void* worker_func(void* arg) {
    usleep(5000000);
    long start = ((long*)arg)[0];
    long end   = ((long*)arg)[1];
    long long sum = 0;
```

```
    long count = 0;
    for (long i = start; i <= end; i++)
        sum += i;

    long thread_id = start == 1 ? 1 : 2;

    if (thread_id == 1) result_T1 = sum;
    else result_T2 = sum;

    pthread_exit(nullptr);
}

/* =======================
  Monitor Thread 函式
======================= */
void* monitor_func(void*) {
    gpio_export(LED_HEARTBEAT);
    gpio_set_direction(LED_HEARTBEAT, "out");

    while (true) {
        int ret = sem_trywait(&sem_done);

        if (ret == 0) break;  // 收到通知 → 停止閃爍

        gpio_set_value(LED_HEARTBEAT, 1);
        usleep(500000);
        gpio_set_value(LED_HEARTBEAT, 0);
        usleep(500000);
    }

    gpio_set_value(LED_HEARTBEAT, 1);
    pthread_exit(nullptr);
}
```

```
/* ======================
  主程式
====================== */
int main() {
   pthread_t t1, t2, t3;
   sem_init(&sem_done, 0, 0);

   // 參數範圍
   long range1[2] = {1, 25000000};
   long range2[2] = {25000001, 50000000};

   // 啟動 Monitor Thread
   pthread_create(&t3, nullptr, monitor_func, nullptr);

   // 啟動 Worker Threads
   pthread_create(&t1, nullptr, worker_func, range1);
   pthread_create(&t2, nullptr, worker_func, range2);

   // 等待 T1、T2 完成
   pthread_join(t1, nullptr);
   pthread_join(t2, nullptr);

   // 通知 T3 停止閃爍
   sem_post(&sem_done);

   // 等待 T3 結束
   pthread_join(t3, nullptr);

   // 彙總結果
   long long total = result_T1 + result_T2;
   cout << "Sum of 1 to 50,000,000 = " << total << endl;

   sem_destroy(&sem_done);
   return 0;
}
```

項目二：main2.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <pthread.h>
#include <unistd.h>

using namespace std;

/* ================
  GPIO 控制函式
================ */

bool file_exists(const string &path) {
   ifstream f(path);
   return f.good();
}

void gpio_export(int pin) {
   if (!file_exists("/sys/class/gpio/gpio" +
to_string(pin))) {
       ofstream exportFile("/sys/class/gpio/export");
       if (exportFile.is_open()) {
           exportFile << pin;
           exportFile.close();
       }
   }
}

void gpio_set_direction(int pin, const string &dir) {
   string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/direction";
   ofstream dirFile(path);
```

```
    if (dirFile.is_open()) {
        dirFile << dir;
        dirFile.close();
    }
}

void gpio_set_value(int pin, int value) {
    string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/value";
    ofstream valFile(path);
    if (valFile.is_open()) {
        valFile << value;
        valFile.close();
    }
}

/* ================
  全域變數
================ */

int Counter = 0;
pthread_mutex_t mutex;

int LED2 = 396;

/* ================
  Thread 函式
================ */

void* count_func(void* arg) {
    for (int i = 0; i < 10000; i++) {
        pthread_mutex_lock(&mutex);
        gpio_set_value(LED2, 1);
```

```
    if (i == 0) {
        cout << "In critical session" << endl;
        usleep(100000);
    }
    Counter++;
    gpio_set_value(LED2, 0);

    if (i == 0) {
                cout << "leave critical session" << endl;
                usleep(100000);
        }
    pthread_mutex_unlock(&mutex);
    }

    pthread_exit(nullptr);
}

/* ================
  主程式
================= */

int main() {
    pthread_t t1, t2;

    // 初始化 Mutex
    pthread_mutex_init(&mutex, nullptr);

    // 初始化 LED2
    gpio_export(LED2);
    gpio_set_direction(LED2, "out");
    gpio_set_value(LED2, 0);

    // 建立兩個 Threads
    pthread_create(&t1, nullptr, count_func, nullptr);
    pthread_create(&t2, nullptr, count_func, nullptr);
```

```
    // 等待完成
    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);

    // 顯示結果
    cout << "Final Counter = " << Counter << endl;

    // 理論正確值： 2 * 10,000 = 20,000
    cout << "Expected Value = 20000" << endl;

    // 清理
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

4. 本次實驗過程說明與解決方法：

本次實驗透過 Pthreads 同時實作資料平行與任務平行架構，並搭配 Semaphore 與 Mutex 控制執行緒同步與資源存取。在項目一中，利用兩個 Worker Threads 分擔大量運算，並以 Monitor Thread 搭配 LED 呈現系統狀態；項目二則透過 Mutex 保護共享計數器，避免資料競爭。實作過程中透過同步機制修正競爭與執行緒控制問題，最終成功完成多執行緒與 GPIO 視覺化整合。

5. 分工：

學號、組員	貢獻比例	工作內容
112590450 李馥亘	25%	小組報告、程式規劃
112590452 林希芸	25%	資料查詢、文書處理
112590454 林妍蓁	25%	實驗環境準備、實驗設計
112590455 吳哲丞	25%	測試與除錯、實驗實作